

Relatório do Projeto

Estrutura de Dados e Algoritmos

Licenciatura em Ciência de Dados - CDA1

Docente: Maria Pinto Albuquerque

Discentes: Camila Sousa nº111017 e Nuno Matos nº111102

04/2023



Função Partitions

- Definir função `partitions`, que recebe dois parâmetros: `file_name` (o nome do arquivo) e `partition_size` (o tamanho das partições que serão criadas).
- Ler o arquivo csv usando a biblioteca `pandas` e armazenar o resultado em `file_name`.
- Selecionar a primeira coluna do arquivo, `iloc[:, 0]`, e converter para lista, `tolist()`, para guardar todos os valores da primeira coluna do arquivo.
- Verificar se `partition_size` é igual a zero. Se for zero, apresentar um erro pois o número é inválido.
- Calcular o número total de partições que serão criadas com base no tamanho do arquivo e no tamanho de cada partição. O operador `//` retorna a divisão inteira, o operador `%` retorna o resto da divisão.
- Criar um gerador que produz as partições do arquivo. O gerador cria uma sequência de fatias do arquivo com base no tamanho de cada partição. A expressão `range(num_partitions)` gera um intervalo de números de `0` a `num_partitions - 1`, que são usados para indexar as fatias do arquivo.
- Iterar sobre as partições geradas pelo gerador.
- A cada iteração, `yield` retorna a partição atual como um objeto gerador. É usado para retornar um valor da função, mas sem terminá-la, pausando-a para que possa ser retomada posteriormente, a partir do ponto em que foi interrompida.

```
import pandas as pd
```

```
def partitions(file_name, partition_size):  
    file_name = pd.read_csv(file_name)  
    file_name = file_name.iloc[:, 0].tolist()  
    if partition_size == 0:  
        raise ValueError("Number invalid")  
    num_partitions = len(file_name) // partition_size + (len(file_name) % partition_size > 0)  
    partitions = (file_name[i * partition_size:(i + 1) * partition_size] for i in range(num_partitions))  
    for partition in partitions:  
        yield partition
```

Função Sorting

Def partitions2:

- Definir uma função chamada **partition2** que recebe uma lista **lst**, um índice inicial **ini** e um índice final **end**.
- Inicializar duas variáveis: **i** recebe o valor do índice inicial – 1, e **pivot** recebe o valor da última posição da lista **lst**, que será usado como pivô no algoritmo de ordenação.
- Iterar sobre os elementos da lista **lst** no intervalo entre **ini** e **end**.
- Verificar se o elemento atual é menor ou igual ao pivô. Se for, incrementa **i** e troca a posição dos elementos **lst[i]** e **lst[j]** trocando os valores sem precisar de uma terceira variável (técnica de "swap")
- Finalizar o algoritmo de ordenação, o pivô é trocado com o elemento na posição **lst[i+1]** e a função retorna o índice da posição do pivô na lista.

Def quicksort:

- Definir uma função chamada **quicksort** que recebe a lista **lst**, um índice inicial **ini**, um índice final **end** e um número máximo de iterações **num_iterations**.
- Adicionar uma condição para verificar se ainda há elementos para ordenar na lista.
- Condição: Se o número de iterações restantes for maior que zero, a função **partition2** é chamada para encontrar o pivô e a lista é dividida em duas partes, que serão ordenadas recursivamente com chamadas à função **quicksort**. Se o número de iterações restantes for igual a zero, a lista é ordenada usando a função **sorted** do *Python*.

Def sorting:

- Definir uma função chamada *sorting* que recebe a lista *lst*, um booleano *is_quicksort*, que indica se se deve usar o algoritmo de ordenação *quicksort* ou não, e um número máximo de iterações *num_iterations*.
- Criar uma cópia da lista *lst* para evitar alterá-la diretamente.

```
def partition2(lst, ini, end):  
    i = ini - 1  
    pivot = lst[end]  
    for j in range(ini, end):  
        if lst[j] <= pivot:  
            i = i + 1  
            lst[i], lst[j] = lst[j], lst[i]  
    lst[i + 1], lst[end] = lst[end], lst[i + 1]  
    return i + 1  
  
def quicksort(lst, ini, end, num_iterations):  
    if ini < end:  
        if num_iterations > 0:  
            partir = partition2(lst, ini, end)  
            num_iterations = num_iterations - 1  
            quicksort(lst, ini, partir - 1, num_iterations)  
            quicksort(lst, partir + 1, end, num_iterations)  
        else:  
            lst[ini:end + 1] = sorted(lst[ini:end + 1])  
  
def sorting(lst, is_quicksort, num_iterations):  
    partition = lst.copy()  
    if is_quicksort:  
        quicksort(partition, 0, len(partition) - 1, num_iterations)  
    else:  
        partition.sort()  
    return partition
```

Para dar print às partições e visualizar se estas estavam ordenadas de forma crescente, prosseguimos com o seguinte código, tomando o tamanho de partições como 20 (por exemplo):

- Iterar em partições do arquivo de dados.
- Gerar partições do arquivo de dados com o tamanho especificado pela variável *partition_size*.
- Adicionar um contador a cada elemento iterado e retornar um tuple com o contador e o elemento em si, usando *enumerate()*. A cada iteração a variável *i* receberá o valor do contador e a variável *partition* receberá a partição atual.
- Chamar uma função chamada *sorting()* que ordena a partição atual em ordem crescente, usando a segunda coluna como chave de ordenação. A variável *sorted_partition* receberá a partição ordenada.
- Imprimir a mensagem "Partition x:", onde x é o número da partição atual, que é obtido adicionando 1 à variável *i*.
- Imprimir a partição atual ordenada.

```
file_name = 'data.csv'
partition_size = 20
for i, partition in enumerate(partitions(file_name, partition_size)):
    sorted_partition = sorting(partition, True, 2)
    print("Partition {}".format(i + 1))
    print(sorted_partition)
```

- Tanto mergesort como quicksort são algoritmos de ordenação eficientes para este objetivo, contudo escolhemos o quicksort, uma vez que é eficiente para conjuntos de dados grandes, podendo haver listas com milhares de valores, por não requerer armazenamento adicional para organizar os dados, e por termos verificado em excel a existência de valores duplicados.

Função Execução

- Definir uma função denominada **execute** que recebe a lista de dados **data**, o tamanho de cada partição **n** e um booleano **ascending** que indica se a ordenação deve ser ascendente ou descendente.
- Criar uma lista vazia para armazenar os resultados
- Iterar sobre todas as partições de tamanho **n** da lista **data**. A função **partitions** é usada para gerar as partições.
- Calcular o tempo de execução da função **sorting** usando a função **time.time**.
- Criar variável **duration** para armazenar o tempo decorrido, que é calculado subtraindo o tempo inicial registrado pelo tempo final também registrado por **time.time()** após fazer sorting (True indica que é ascending e 2 o nº de iterações).
- Determinar o valor máximo e mínimo da partição de acordo com a ordem especificada pelo parâmetro **ascending**: se for verdadeiro, o valor máximo é o maior valor da partição e o valor mínimo é o menor valor da partição, caso contrário, o valor máximo é o menor valor da partição e o valor mínimo é o maior valor da partição.
- Calcular o número máximo de iterações que o algoritmo **quicksort** pode fazer para ordenar a partição
- Adicionar um tuple com os resultados da partição à lista **results**, que contém o tempo de execução, o valor máximo, o valor mínimo e o número máximo de iterações
- Criar um dicionário com as quatro colunas do DataFrame, **duration**, **max_val**, **min_val** e **num_iterations**, e associa cada uma dessas colunas com uma lista correspondente dos valores retornados pela função **execute()** para cada iteração. **[r[0] for r in results]** cria uma lista com todos os valores do primeiro elemento do tuple retornadas pela função **execute()**, e assim por diante para cada uma das outras colunas. Assim, cada chave do dicionário representa uma coluna do DataFrame, e cada lista representa os valores para essa coluna.
- A linha **df = pd.DataFrame(df)** cria o DataFrame a partir do dicionário **df**. A função **pd.DataFrame()** do pandas transforma o dicionário em um DataFrame com as colunas especificadas e seus valores correspondentes.

- *Print(df)* exibe o DataFrame
- Retornar a lista de resultados

```
def execute(data, n, ascending):
    results = []
    for partition in partitions(data, n):
        start_time = time.time()
        sorting(partition, True, 2)
        duration = time.time() - start_time
        if ascending:
            max_val, min_val = max(partition), min(partition)
        else:
            max_val, min_val = min(partition), max(partition)
        num_iterations = len(partition) // 2
        results.append((duration, max_val, min_val, num_iterations))

    df = {
        'duration': [r[0] for r in results],
        'max_val': [r[1] for r in results],
        'min_val': [r[2] for r in results],
        'num_iterations': [r[3] for r in results],
    }

    df = pd.DataFrame(df)
    print(df)

    return results
```

Experiências (Código):

- Definir uma lista de valores *n* para serem usados na execução do algoritmo, cada valor de "n" representa o número de observações em cada partição.
- Criar uma lista vazia para armazenar os tempos de execução.
- Iterar sobre os valores de "n" definidos anteriormente.
- Chamar a função *execute* para ordenar os dados do arquivo *data.csv* (*file_name*) usando partições de tamanho *n* e ordenando de forma ascendente. Armazenar os resultados na lista *results*.
- Calcular o tempo total de execução somando o tempo de execução de cada partição contido na lista *results*. A lista é usada para aceder apenas o tempo de

execução de cada partição. O uso do caractere `_` é uma convenção que indica que aquela variável não será utilizada, isto é, os outros três elementos do tuple são ignorados.

- Adicionar o tempo total de execução da ordem de *n* observações à lista *execution_times*.
- Gerar um gráfico de linha que mostra a relação entre o número de observações por partição (eixo x) e o tempo total de execução (eixo y).
- Definir os rótulos dos eixos x e y
- Mostrar o gráfico gerado.
- *duration*, *max_val*, *min_val* e *num_iterations* são variáveis que receberão os valores retornados pela função *execute()* em cada iteração.

Quando testamos o código apesar de nos ser apresentado o gráfico deparámo-nos com erros deste género *"backend_mod.show = manager_class.pyplot_show"* . Ao pesquisar como corrigir, percebemos que provavelmente o backend usado não tinha o método `pyplot_show` definido corretamente ou faltava alguma importação necessária. Uma solução para esse problema seria escolher um backend diferente para a geração de gráficos, e por isso fizemos `matplotlib.use(TkAgg)` para que o matplotlib usasse o Tkinter (backend que provavelmente já tinha o método `pyplot_show` definido corretamente/ as importações necessárias estavam corretas) para exibir os gráficos.

```
import matplotlib
import matplotlib.pyplot as plt
matplotlib.use('TkAgg')
```

```
n_values = [10, 75, 250, 500, 1000]
execution_times = []
for n in n_values:
    results = execute(file_name, n, True)
    total_time = sum([duration for duration, _, _, _ in results])
    execution_times.append(total_time)

plt.plot(n_values, execution_times)
plt.xlabel('Number of observations per partition')
plt.ylabel('Execution time (s)')
plt.show()
```


Experiências

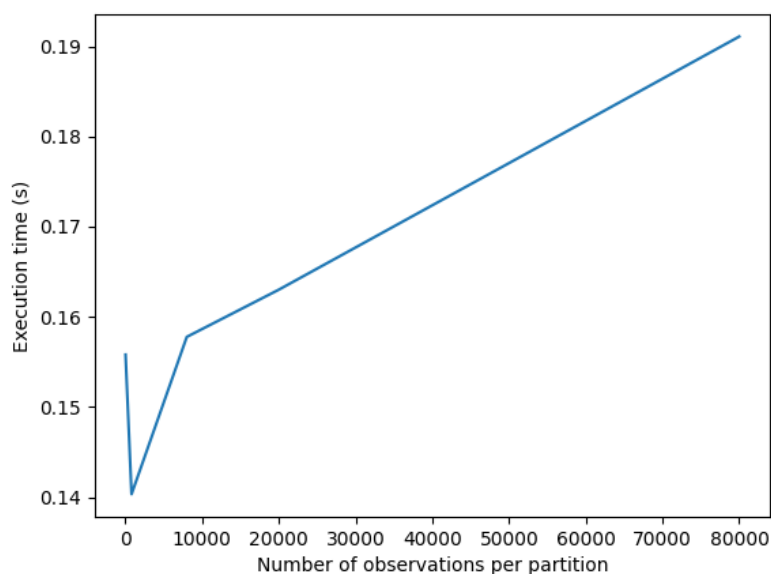
Ao realizar várias experiências com diferentes valores de n (número de observações por partição), podemos analisar como o tempo de execução é afetado pelo tamanho da partição.

Em geral, espera-se que o tempo de execução aumente à medida que o tamanho da partição aumenta, uma vez que a ordenação de grandes conjuntos de dados requer mais tempo de processamento. O tempo de execução do QuickSort é proporcional a $O(n \log n)$ e por isso o tempo total de execução aumenta de maneira exponencial à medida que o tamanho das partições aumenta. No entanto, também podemos observar que, em alguns casos, o tempo de execução diminui ligeiramente à medida que o tamanho da partição aumenta, devido à otimização de leitura de disco e armazenamento do sistema operacional.

Portanto, é importante realizar experimentos com diferentes tamanhos de partição para determinar o tamanho ideal para o conjunto de dados, procurando um equilíbrio entre o tamanho da partição e o tempo de execução de maneira a obter o melhor desempenho possível.

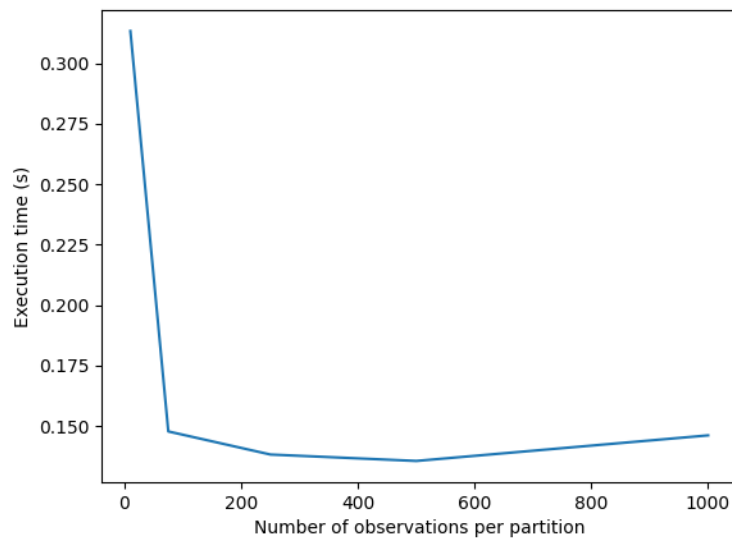
Foram realizadas as seguintes experiências com diferentes intervalos de *n_values*:

```
n_values = [20, 800, 8000, 20000, 80000]
```



Como a complexidade do algoritmo é $O(n \log n)$, espera-se que o tempo de execução também aumente de forma logarítmica à medida que n aumenta. O aumento linear deve-se possivelmente pelo número de dados por partição estar a aumentar numa taxa constante. Indica que pode haver um equilíbrio entre precisão e velocidade em um determinado ponto, dependendo dos recursos disponíveis.

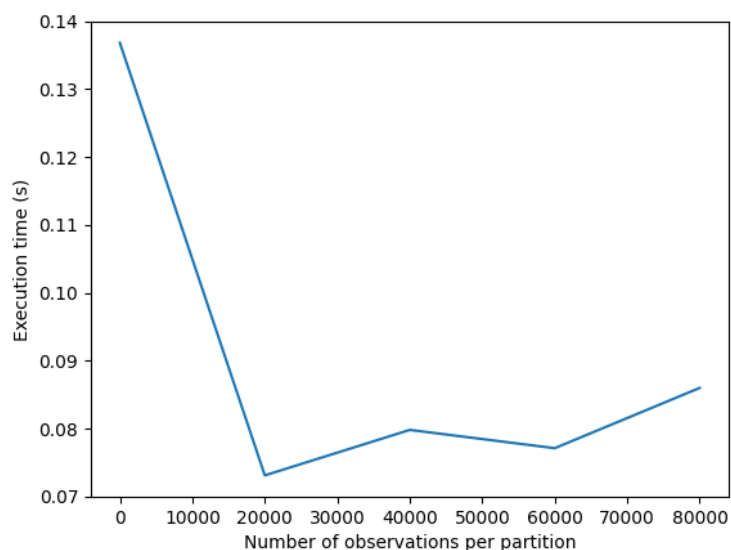
```
n_values = [10, 75, 250, 500, 1000]
```



Neste experimento o gráfico desce em vez de crescer, talvez porque a quantidade de dados por partição (n) seja muito pequena e a complexidade do algoritmo não está a ser suficiente para que o tempo de execução cresça à medida que o número de dados aumenta.

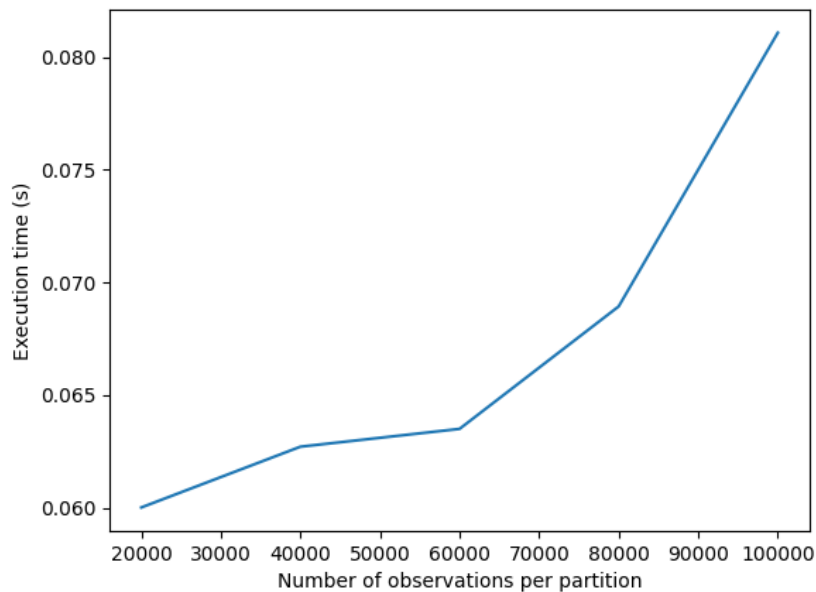
De uma maneira geral fizemos de 20 000 em 20 000 até 100 000:

```
n_values = list(range(1, 100000, 20000))
```



De seguida fizemos:

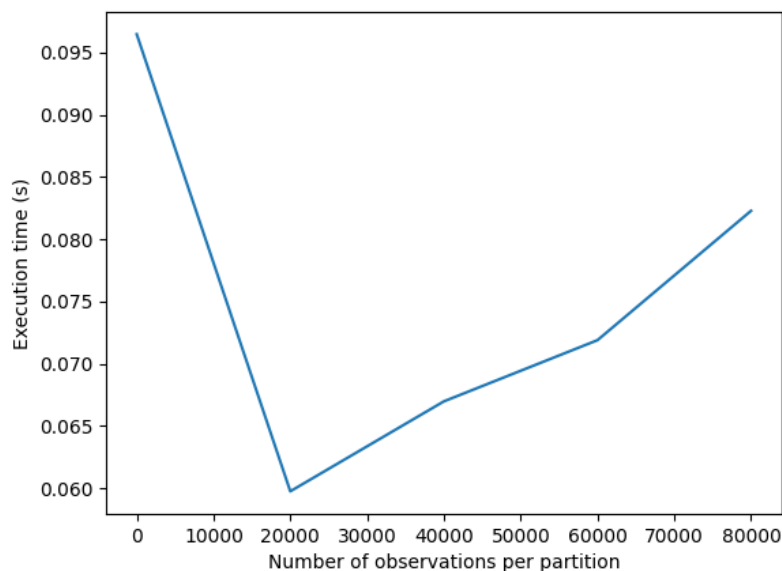
```
n_values = list(range(20000, 100000, 20000))
```



Nesse experimento, podemos ver que o tempo total de execução aumenta drasticamente à medida que o tamanho das partições aumenta. Isso ocorre porque, para partições maiores, o algoritmo QuickSort precisa executar um número maior de iterações recursivas para ordenar corretamente cada partição.

Para analisar num intervalo de números mais curtos fizemos:

```
n_values = list(range(5000, 30000, 5000))
```



Conclusões

Considerando as experiências realizadas, concluímos que o tamanho de partição(n) que minimiza o tempo de execução do código é 20 000. É importante lembrar que a escolha de um valor para n vai depender do objetivo do projeto, por exemplo, se são projetos em que a precisão é mais importante do que a velocidade.

Em suma, esse código é útil para executar e avaliar o desempenho do algoritmo de ordenação Quicksort em grandes conjuntos de dados armazenados em arquivos CSV, permitindo ajustar o tamanho das partições e o número máximo de iterações do Quicksort. O gráfico gerado também ajuda a visualizar como o desempenho do algoritmo é afetado pelo tamanho da partição.

Bibliografia

- Matos, B. R. S. de. (2022, May 27). Merge e Quick Sort: Entenda Qual É o Melhor Algoritmo. Alura. Retrieved April 16, 2023, from <https://www.alura.com.br/artigos/merge-quick-sort-qual-o-melhor-algoritmo>
- Lima, M. de, & Peres, L. (2021, September 14). 10 funções mais usadas para manipular dataframes no pandas. Insight Data Science Lab. Retrieved April 16, 2023, from <https://www.insightlab.ufc.br/10-funcoes-mais-usadas-para-manipular-dataframes-no-pandas/>
- Rajinikanth. (n.d.). Data Structures. Data Structures Tutorials - Quick Sort Algorithm with an example. Retrieved April 16, 2023, from http://www.btechsmartclass.com/data_structures/quick-sort.html
- Popović, O. (2020, May 31). Quicksort in python. Stack Abuse. Retrieved April 16, 2023, from <https://stackabuse.com/quicksort-in-python/>
- Backends. Backends - Matplotlib 3.7.1 documentation. (n.d.). Retrieved April 16, 2023, from <https://matplotlib.org/stable/users/explain/backends.html>
- My plot in ipython does not show with pyplot.show(). (s.d.). Stack Overflow. <https://stackoverflow.com/questions/7534453/my-plot-in-ipython-does-not-show-with-pyplot-show>
- Using Matplotlib with tkinter (TkAgg). (s.d.). Stack Overflow. <https://stackoverflow.com/questions/35829961/using-matplotlib-with-tkinter-tkagg>
- Quick Sort vs Merge Sort - GeeksforGeeks. (s.d.-b). GeeksforGeeks. <https://www.geeksforgeeks.org/quick-sort-vs-merge-sort/>
- Is there a standardized method to swap two variables in Python? (s.d.). Stack Overflow. <https://stackoverflow.com/questions/14836228/is-there-a-standardized-method-to-swap-two-variables-in-python>
- Time â Time access and conversions. (s.d.). Python documentation. <https://docs.python.org/3/library/time.html>

- How do I select a subset of a DataFrame? — pandas 2.0.0 documentation.
(s.d.). pandas - Python Data Analysis Library.
https://pandas.pydata.org/docs/getting_started/intro_tutorials/03_subset_data.html

- Python iloc() function - All you need to know! - AskPython. (s.d.). AskPython.
<https://www.askpython.com/python/built-in-methods/python-iloc-function>

- S, L. (2021, 3 de fevereiro). Como importar arquivos CSV no Pandas (Python).
Python Academy. <https://pythonacademy.com.br/blog/importar-csv-no-pandas>

- Pankaj. (2022, 3 de agosto). How To Convert a NumPy Array to List in Python.
DigitalOcean | The Cloud for Builders.
<https://www.digitalocean.com/community/tutorials/python-convert-numpy-array-to-list>

- Geradores em Python. (s.d.). giovannireisnunes.
<https://giovannireisnunes.wordpress.com/2019/03/01/geradores-em-python/>